



# **K-NEAREST NEIGHBORS (KNN)**

**APRENDIZAGEM DE MÁQUINA**

**CST em Desenvolvimento de Software Multiplataforma**



**PROF. Me. TIAGO A. SILVA**



# O QUE É O ALGORITMO KNN?

- O algoritmo K-Nearest Neighbors (KNN), ou K-Vizinhos Mais Próximos, é um dos mais intuitivos e fundamentais algoritmos de aprendizado de máquina.
- Utilizado tanto para tarefas de classificação quanto de regressão, sua simplicidade e eficácia o tornam um ponto de partida popular para muitos problemas em ciência de dados.
- A lógica central do KNN se baseia em um conceito simples: **"diga-me com quem andas e te direi quem és"**.
  - Essencialmente, um novo dado é classificado com base na classe da maioria de seus vizinhos mais próximos.

# COMO FUNCIONA O KNN?

- O KNN é considerado um algoritmo de "aprendizado preguiçoso" (lazy learning), pois, ao contrário de muitos outros algoritmos, ele não constrói um modelo durante a fase de treinamento.
- Em vez disso, ele simplesmente armazena todo o conjunto de dados de treinamento. A verdadeira "aprendizagem" ocorre no momento da predição.

# COMO FUNCIONA O KNN?

- O processo pode ser dividido nos seguintes etapas:

## 1) Escolha do Número de Vizinhos (K):

- O primeiro passo é definir o valor de 'K', que representa o número de vizinhos que serão consultados para classificar um novo ponto de dados. A escolha de 'K' é crucial para o desempenho do algoritmo.

## 2) Cálculo das Distâncias:

- Quando um novo dado precisa ser classificado, o KNN calcula a distância entre este novo ponto e todos os outros pontos no conjunto de dados de treinamento. Existem várias métricas de distância que podem ser utilizadas, sendo as mais comuns: **Distância Euclidiana e Distância de Manhattan**

# ETAPA2: CALCULO DA DISTÂNCIA - FUNÇÕES DE DISTÂNCIA

- **Distância Euclidiana:**

- A distância em linha reta entre dois pontos. É a métrica mais utilizada e funciona bem em muitos cenários:

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

- **Distância de Manhattan:**

- A soma das diferenças absolutas entre as coordenadas dos pontos. É útil quando os dados possuem alta dimensionalidade:

$$d(p, q) = \sum_{i=1}^n |p_i - q_i|$$

## ETAPA 3: IDENTIFICAÇÃO DOS K-VIZINHOS MAIS PRÓXIMOS

- Identificação dos K Vizinhos Mais Próximos:
  - Após calcular todas as distâncias, o algoritmo seleciona os 'K' pontos de dados de treinamento que possuem as menores distâncias em relação ao novo ponto.

## ETAPA 4: CLASSIFICAÇÃO

- Votação (Classificação) ou Média (Regressão):
  - **Para Classificação:** O novo ponto de dados é atribuído à classe que aparece com mais frequência entre os 'K' vizinhos selecionados (voto majoritário).
  - **Para Regressão:** O valor previsto para o novo ponto de dados é a média dos valores dos 'K' vizinhos mais próximos.

# QUAL A IMPORTÂNCIA DO VALOR DE K?

- A escolha de 'K' tem um impacto significativo no resultado.
  - Um 'K' pequeno torna o modelo mais sensível a ruídos e outliers, podendo levar a um sobreajuste (overfitting).
  - Um 'K' grande pode suavizar demais os limites de decisão, resultando em um subajuste (underfitting), onde o modelo não captura a complexidade dos dados.
- Geralmente, um valor ímpar para 'K' é escolhido para evitar empates na votação para problemas de classificação. A determinação do 'K' ideal muitas vezes envolve a experimentação e o uso de técnicas como a validação cruzada para avaliar o desempenho do modelo com diferentes valores de 'K'.



# APLICAÇÕES DO KNN

- A versatilidade do KNN permite sua aplicação em diversas áreas, incluindo:
  - Sistemas de Recomendação: Sugerir produtos, filmes ou músicas com base nas preferências de usuários com gostos semelhantes.
  - Reconhecimento de Padrões: Classificação de imagens, como o reconhecimento de dígitos manuscritos ou a identificação de objetos.
  - Detecção de Anomalias: Identificar transações fraudulentas em sistemas financeiros ou detectar falhas em equipamentos industriais.
  - Bioinformática: Classificar amostras genéticas ou prever a função de proteínas.

## VANTAGENS E DESVANTAGENS

*Cada ferramenta, ao seu propósito*

# VANTAGENS DO KNN

- Simplicidade e Intuitividade: Fácil de entender e implementar.
- Flexibilidade: Pode ser usado tanto para classificação quanto para regressão.
- Não Paramétrico: Não faz suposições sobre a distribuição dos dados, o que o torna útil para problemas complexos e não lineares.
- Adaptação Rápida: Novos dados de treinamento podem ser adicionados facilmente sem a necessidade de retreinar um modelo complexo.

# DESVANTAGENS DO KNN

- **Custo Computacional:** A necessidade de calcular a distância para todos os pontos de treinamento a cada nova predição pode ser lenta e consumir muitos recursos, especialmente com grandes conjuntos de dados.
- **Sensibilidade a Atributos Irrelevantes:** A presença de muitos atributos irrelevantes pode mascarar a importância dos atributos significativos, prejudicando a precisão do cálculo de distância.
- **Maldição da Dimensionalidade:** Em espaços com muitas dimensões, a noção de "vizinhança" se torna menos significativa, e o desempenho do KNN tende a diminuir.
- **Necessidade de Normalização dos Dados:** Atributos em escalas diferentes podem influenciar desproporcionalmente o cálculo da distância. Portanto, a normalização dos dados (colocar todos os atributos na mesma escala) é uma etapa de pré-processamento crucial.

# IMPLEMENTAÇÃO DO CÓDIGO

*Implementando KNN com Python*

# CÉLULA 1: IMPORTANDO AS BIBLIOTECAS

● ● ● Aula 4 - KNN - analyse\_knn\_tumores.ipynb

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from collections import Counter
```

# CARREGAMENTO DOS DADOS

*Carregando o dataset*

# CÉLULA 2: CARREGANDO OS DADOS DO DATASET



Aula 4 - KNN - analise\_knn\_tumores.ipynb

```
1 # Carrega os dados do CSV. Delimiter=', ' indica que as colunas são separadas por vírgula.
2 # skiprows=2 para pular as duas linhas de cabeçalho/comentário.
3 dataset = np.loadtxt("tumores.csv", delimiter=',', skiprows=2)
4
5 # Separa as colunas de características (features) e a coluna de rótulos (labels)
6 # X_train recebe todas as linhas, mas apenas as duas primeiras colunas (índice 0 e 1)
7 X_train = dataset[:, :2]
8
9 # y_train recebe todas as linhas, mas apenas a última coluna (índice 2)
10 # .astype(int) para garantir que as classes sejam tratadas como inteiros
11 y_train = dataset[:, 2].astype(int)
12
13 print("Amostra das características (X_train):")
14 print(X_train[:5])
15 print("\nAmostra dos rótulos (y_train):")
16 print(y_train[:5])
```



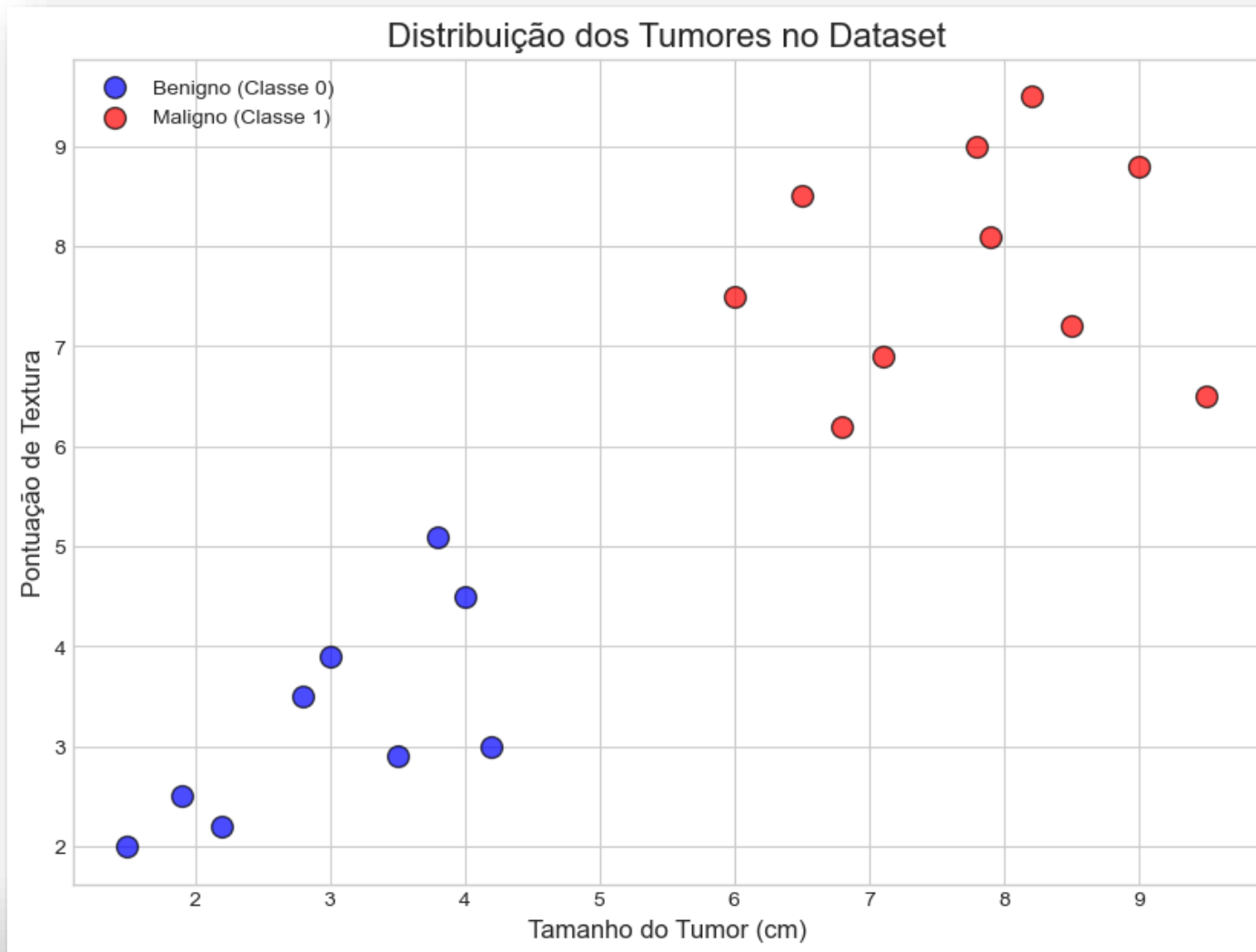
# CÉLULA 3: VISUALIZAÇÃO DOS DADOS DE TREINAMENTO



Aula 4 - KNN - analyse\_knn\_tumores.ipynb

```
1 plt.style.use('seaborn-v0_8-whitegrid')
2 fig, ax = plt.subplots(figsize=(10, 7))
3
4 # Plota os pontos Benignos (Classe 0)
5 ax.scatter(X_train[y_train == 0, 0], X_train[y_train == 0, 1],
6           c='blue', s=100, edgecolor='k', alpha=0.7, label='Benigno (Classe 0)')
7
8 # Plota os pontos Malignos (Classe 1)
9 ax.scatter(X_train[y_train == 1, 0], X_train[y_train == 1, 1],
10          c='red', s=100, edgecolor='k', alpha=0.7, label='Maligno (Classe 1)')
11
12 ax.set_title('Distribuição dos Tumores no Dataset', fontsize=16)
13 ax.set_xlabel('Tamanho do Tumor (cm)', fontsize=12)
14 ax.set_ylabel('Pontuação de Textura', fontsize=12)
15 ax.legend()
16 plt.show()
```

# VISUALIZAÇÃO DOS DADOS DE TREINAMENTO



# IMPLEMENTANDO O ALGORITMO KNN

*Implementação usando apenas Numpy e  
Matplotlib*

# CÉLULA 4: FUNÇÃO DA DISTÂNCIA EUCLIDIANA

$$d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$



Aula 4 - KNN - analyse\_knn\_tumores.ipynb

```
1 def calcular_distancia_euclidiana(ponto1, ponto2):  
2     """Calcula a distância euclidiana entre dois pontos."""  
3     return np.sqrt(np.sum((ponto1 - ponto2)**2))
```

# CÉLULA 4: FUNÇÃO PARA ENCONTRAR VIZINHOS



Aula 4 - KNN - analyse\_knn\_tumores.ipynb

```
5 def encontrar_vizinhos(X_train, y_train, ponto_teste, k):
6     """Encontra os k vizinhos mais próximos de um ponto de teste."""
7     distancias = []
8     for i, ponto_treino in enumerate(X_train):
9         dist = calcular_distancia_euclidiana(ponto_treino, ponto_teste)
10        distancias.append((dist, y_train[i]))
11
12    # Ordena a lista de distâncias em ordem crescente
13    distancias.sort(key=lambda x: x[0])
14
15    # Retorna os k vizinhos mais próximos (seus rótulos)
16    vizinhos = [distancia[1] for distancia in distancias[:k]]
17    return vizinhos
```

# CÉLULA 4: PREVER CLASSIFICAÇÃO



Aula 4 - KNN - analyse\_knn\_tumores.ipynb

```
19 def prever_classificacao(vizinhos):
20     """Faz a previsão com base no voto majoritário dos vizinhos."""
21     # Conta a ocorrência de cada classe nos vizinhos
22     contagem_votos = Counter(vizinhos)
23     # Retorna a classe mais comum
24     previsao = contagem_votos.most_common(1)[0][0]
25     return previsao
```

## REALIZANDO PREVISÕES

*Passando um dado e observando como será  
classificado*

# CÉLULA 5: REALIZANDO PREVISÕES

Aula 4 - KNN - analise\_knn\_tumores.ipynb

```
1 # Ponto que queremos classificar
2 novo_tumor = np.array([5.5, 6.0])
3 k = 5
4
5 # Encontra os vizinhos e faz a previsão
6 vizinhos_proximos = encontrar_vizinhos(X_train, y_train, novo_tumor, k)
7 previsao_final = prever_classificacao(vizinhos_proximos)
8
9 # Mapeia o resultado para um texto explicativo
10 resultado_texto = "Benigno" if previsao_final == 0 else "Maligno"
11
12 print(f"O novo tumor tem as características: {novo_tumor}")
13 print(f"O valor de 'k' escolhido foi: {k}")
14 print(f"As classes dos vizinhos mais próximos são: {vizinhos_proximos}")
15 print("-----")
16 print(f"--> Previsão Final: O novo tumor é classificado como {resultado_texto} (Classe {previsao_final})")
```



## PLOTANDO OS GRÁFICOS

*Apresenta a classificação de acordo com a posição no plot*

# CÉLULA 6: IMPLEMENTAÇÃO DO PLOT DA PREVISÃO



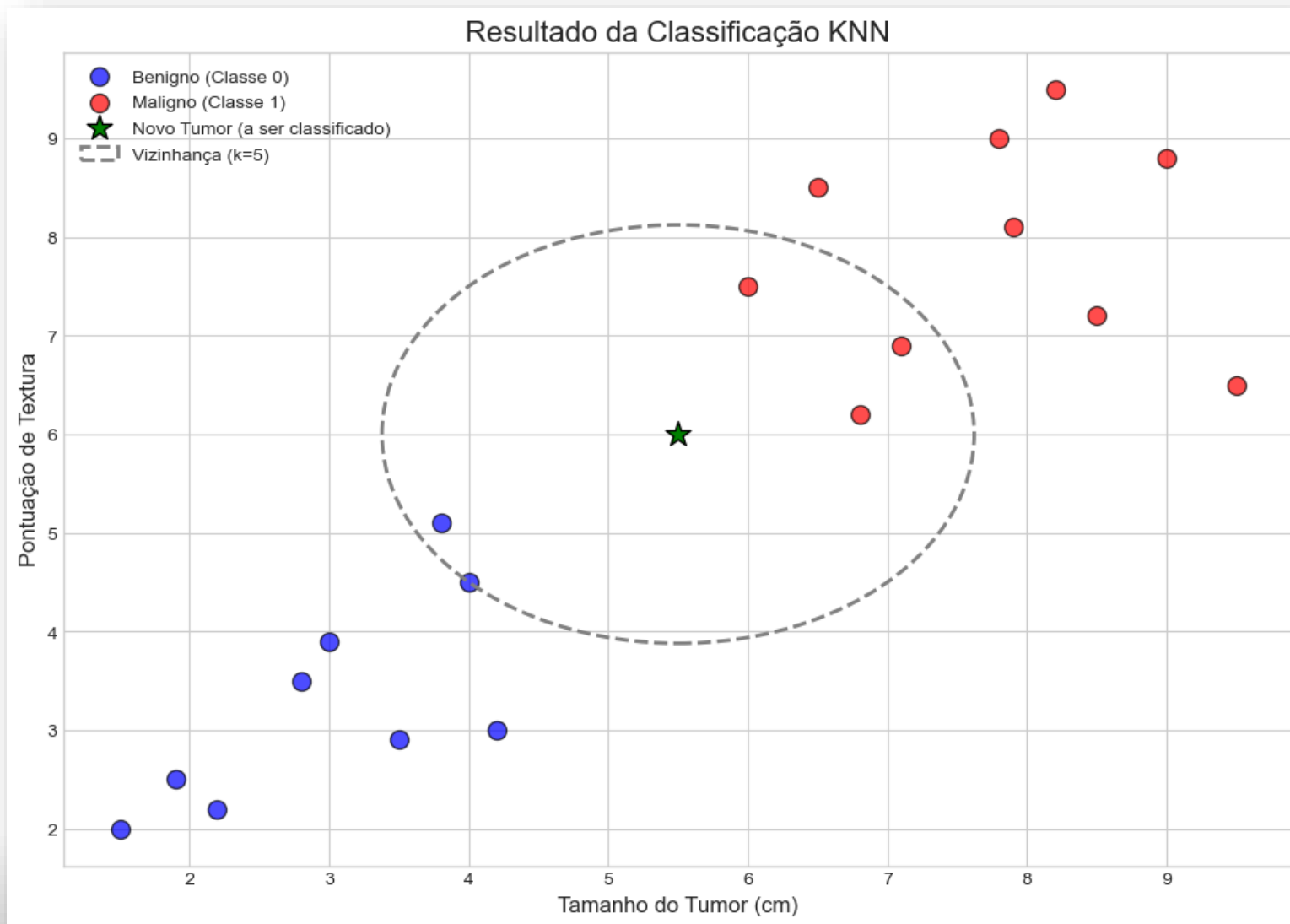
```
1  fig, ax = plt.subplots(figsize=(12, 8))
2
3  # Plotando os pontos de dados de treinamento
4  ax.scatter(X_train[y_train == 0, 0], X_train[y_train == 0, 1],
5             c='blue', s=100, edgecolor='k', alpha=0.7, label='Benigno (Classe 0)')
6  ax.scatter(X_train[y_train == 1, 0], X_train[y_train == 1, 1],
7             c='red', s=100, edgecolor='k', alpha=0.7, label='Maligno (Classe 1)')
8
9  # Plotando o novo ponto de dados a ser classificado
10 ax.scatter(novo_tumor[0], novo_tumor[1], c='green', s=200,
11            edgecolor='k', marker='*', label='Novo Tumor (a ser classificado)')
12
13 # Destaque para os vizinhos mais próximos
14 indices_vizinhos = np.argsort([calcular_distancia_euclidiana(p, novo_tumor) for p in X_train])[:k]
15 pontos_vizinhos = X_train[indices_vizinhos]
```

# CÉLULA 6: IMPLEMENTAÇÃO DO PLOT DA PREVISÃO



```
17 # Desenhando um círculo que engloba os vizinhos
18 distancia_maxima = calcular_distancia_euclidiana(novo_tumor, pontos_vizinhos[-1])
19 circulo = plt.Circle(novo_tumor, distancia_maxima, color='gray', fill=False,
20                      linestyle='--', linewidth=2, label=f'Vizinhança (k={k})')
21 ax.add_artist(circulo)
22
23 # Configurações do gráfico
24 ax.set_title('Resultado da Classificação KNN', fontsize=16)
25 ax.set_xlabel('Tamanho do Tumor (cm)', fontsize=12)
26 ax.set_ylabel('Pontuação de Textura', fontsize=12)
27 ax.legend(loc='upper left')
28 ax.grid(True)
29 plt.show()
```

# A ESTRELA É O DADO DA PREVISÃO COM SUA VIZINHANÇA



# OBRIGADO!

- Encontre este **material on-line** em:
  - Slides: Google Classroom
- Em caso de **dúvidas**, entre em contato:
  - **Prof. Tiago:** [tiago.silva@fatecjahu.edu.br](mailto:tiago.silva@fatecjahu.edu.br)

